search_api.py ×

```
import requests

def search(search_term='luke'):
  base_url = 'https://swapi.dev/api/people/?search='
  search_url = f'{base_url}{search_term}'
  resp = requests.get(search_url)
  resp_json = resp.json()
  if resp_json.get('results'):
    return resp.json()['results'][0]
  else:
    return None

if __name__ == '__main__':
  import pprint

  character = search()
  pprint.pprint(character)
```

12% (2:1)

Terminal ×

```
(base) codio@cabinetchef-igloofocus:~/workspace$
```

Guide ×

Collapse

Parsing the Name and Planet

Parsing the Name

Activate the virtual environment.

```
conda activate sw
```

We need the name from the JSON data. Using the `get()` method, store the value of `'name'` and return it. The name is stored as a string in the dictionary.

```
def parse_name(person):
  name = person.get('name')
  return name
```

Parsing the Planet

Unlike the name, the planet is not stored as a string. Instead, it is another URL for the Star Wars API. First, extract the planet URL which is stored under the `'homeworld'` key. Then make another API call with the `requests` package. Convert the response to JSON and return the value associated with the `'name'` key, which is a string of the planet's name.

```
def parse_planet(person):
  planet_url = person.get('homeworld')
  resp = requests.get(planet_url)
  planet = resp.json().get('name')
  return planet
```

Testing the Functions

In the testing section at the bottom, change the character search to be the default. Then create the `name` variable and pass it `character`. Print `name` and you should see `'Luke Skywalker'`.

```
if __name__ == '__main__':
  import pprint

  character = search()
  name = parse_name(character)

  pprint.pprint(name)
```

In the terminal, run the script to test your work.

```
python sw_characters/search_api.py
```

Next, test the `parse_planet` function. Create the variable `planet` and set it to the function call `parse_planet`. Use `character` as the parameter. You should see `'Tatooine'` as the output.

```
if __name__ == '__main__':
  import pprint

  character = search()
  name = parse_name(character)
  planet= parse_planet(character)

  pprint.pprint(planet)
```

In the terminal, run the script to test your work.

```
python sw_characters/search_api.py
```

You should notice that printing the name of the planet takes longer than printing the name. That is because printing the name requires only one API call. Printing the planet name requires two API calls — one for the general character information and another for the planet information. We will see how these API calls start to add up and affect the performance of the script.

▼ **Code**

Your code should look like this:

```
import requests

def search(search_term='luke'):
  base_url = 'https://swapi.dev/api/people/?search='
  search_url = f'{base_url}{search_term}'
  resp = requests.get(search_url)
  resp_json = resp.json()
  if resp_json.get('results'):
    return resp.json()['results'][0]
  else:
    return None

def parse_name(person):
  name = person.get('name')
  return name

def parse_planet(person):
  planet_url = person.get('homeworld')
  resp = requests.get(planet_url)
  planet = resp.json().get('name')
  return planet

if __name__ == '__main__':
  import pprint

  character = search()
  name = parse_name(character)
  planet= parse_planet(character)

  pprint.pprint(planet)
```

Deactivate the virtual environment.

```
conda deactivate
```

Lab Question

Fill in the blanks below.

- The default package repository for Conda is Anaconda.
- The default package repository for Poetry is PyPI.

The correct answers are:

- The default package repository for Conda is **Anaconda**.
- The default package repository for Poetry is **PyPI**.

Conda can use `pip` to install packages from PyPI, but its default package repository is Anaconda. Both Poetry and Conda can install packages hosted on a version control service like GitHub or install a local package. However, neither package manager defaults to this.

Check It!

Next